Caso 5 · Machine Learning

Eficiencia Energética en Edificios Verdes

Análisis completo de datos para predecir el consumo de calefacción de edificios a partir de sus características físicas, usando regresión lineal y regresión logística.

📦 768 edificios analizados 🔢 8 características por edificio

🎯 2 tareas: regresión y clasificación 🌿 Fuente: UCI Machine Learning Repository

🗺️ Flujo General del Proyecto

El proyecto sigue una metodología estándar de ciencia de datos. Cada paso depende del anterior:

1. Preparar entorno → 2. Cargar datos → 3. Explorar (EDA) →
4. Dividir datos → 5. Preprocesar → 6. Entrenar modelos →
7. Evaluar resultados

> 💡 **¿Por qué este orden?** No se puede entrenar un modelo sin entender primero los datos (paso 3), ni entrenar con datos sucios sin preprocesarlos (paso 5). El orden garantiza resultados confiables y reproducibles.

⚙️ Sección 1 — Preparación del Entorno

Antes de tocar los datos, el notebook verifica que la computadora tiene todo lo necesario para correr el análisis sin errores ni diferencias entre máquinas distintas.

¿Qué se verifica?

✅ **Python 3.12.12+**
El lenguaje de programación. Versiones antiguas pueden tener errores de sintaxis.

✅ **RAM ≥ 12.67 GB**
Memoria suficiente para cargar y procesar los datos sin que la computadora se congele.

✅ **CPU ≥ 2 núcleos**
Procesador mínimo para que los algoritmos de machine learning corran en tiempo razonable.

⚠️ **Sistema Operativo: Linux**
El entorno original era Linux; en Mac/Windows puede funcionar igual pero se avisa.

Librerías utilizadas

Librería	¿Para qué sirve?
`pandas`	Manipular tablas de datos (como Excel en Python)
`numpy`	Cálculos matemáticos y matrices
`matplotlib` / `seaborn`	Crear todos los gráficos
`scikit-learn`	Todos los algoritmos de machine learning (modelos, métricas, preprocesamiento)
`psutil`	Leer información del hardware (RAM, CPU)

💡 Se fija la **semilla aleatoria en 42** (`random_state=42`). Esto hace que cualquier operación que use azar (como dividir los datos) siempre dé el mismo resultado, sin importar en qué computadora se corra.

📥 Sección 2 — Cargado de Datos

Paso 2 de 7

El dataset proviene del repositorio de la UCI (Universidad de California, Irvine). Se descarga automáticamente desde internet, se descomprime y se carga en memoria. Así todos trabajan exactamente con los mismos datos.

¿Qué contiene el dataset?

768
Edificios simulados

8
Características físicas

0
Valores faltantes

2
Variables objetivo

Variables del dataset

Variable	Nombre descriptivo	Tipo	Qué representa
`X1`	Relative_Compactness	**Numérica**	Qué tan "compacto" es el edificio (0.62–0.98). Más compacto = más eficiente en general.
`X2`	Surface_Area	**Numérica**	Área total de superficie exterior en m² (514–808 m²).

`X3`	Wall_Area	**Numérica**	Área de paredes en m² (245–416 m²).
`X4`	Roof_Area	**Numérica**	Área del techo en m² (110–220 m²).
`X5`	Overall_Height	**Numérica**	Altura del edificio: 3.5 m (1 piso) o 7 m (2 pisos).
`X6`	Orientation	**Categórica**	Orientación de la fachada principal: Norte / Sur / Este / Oeste.
`X7`	Glazing_Area	**Numérica**	Ratio de ventanales (0 = sin ventanas, 0.4 = 40% de la superficie son vidrio).
`X8`	Glazing_Area_Distribution	**Categórica**	Cómo están distribuidos los ventanales: uniforme, concentrados al Norte, Sur, etc.
`Y1`	Heating_Load	🎯 **Target regresión**	Energía necesaria para calentar el edificio (kWh/m²). Este es el número que queremos predecir.
`Y2`	Cooling_Load	Numérica	Energía para enfriarlo. No se usa como variable objetivo en este estudio.

Variable creada: `Eficiente` (para clasificación)

Para poder clasificar edificios, se crea una variable binaria nueva a partir de `Heating_Load`:

📏 **Umbral = mediana de Heating_Load = 18.95 kWh/m²**

- **Eficiente = 1** → Heating_Load ≤ 18.95 (bajo consumo, buen diseño)
- **Eficiente = 0** → Heating_Load > 18.95 (alto consumo, diseño descartable)

Resultado: **384 edificios eficientes / 384 no eficientes** — clases perfectamente balanceadas (50/50).

🔍 Sección 3 — Análisis Exploratorio de Datos (EDA)

Paso 3 de 7

Antes de entrenar cualquier modelo, hay que *conocer los datos*. El EDA responde preguntas clave: ¿hay datos raros? ¿las variables son útiles? ¿alguna puede confundir al modelo? Esta sección es la más larga porque es la más importante.

3.1 — Análisis Univariado (una variable a la vez)

Se analiza cada variable por separado para entender su comportamiento.

📊 Histogramas con curva KDE (variables numéricas)

Qué muestra: cuántos edificios tienen cada valor de cada característica. La curva suavizada (KDE) indica la forma de la distribución.

Qué buscar: el *sesgo* (skew). Si la cola es muy larga hacia un lado, la distribución está sesgada y puede necesitar transformación matemática.

Variable	Sesgo	Interpretación
Relative_Compactness	~0	Simétrica — No necesita transformación
Surface_Area	~0	Simétrica
Wall_Area	0.53	Sesgo moderado — Aceptable, sin acción
Roof_Area	~0	Simétrica

Overall_Height	~0	**Simétrica**
Glazing_Area	~0	**Simétrica**
Heating_Load (target)	~0	**Simétrica** — No necesita log10

📋 Gráficos de barras (variables categóricas)

Qué muestra: cuántos edificios hay en cada categoría de *Orientation* y *Glazing_Area_Distribution*.

Resultado:

- **Orientation:** 192 edificios exactamente en cada punto cardinal (Norte/Sur/Este/Oeste) → perfectamente balanceada.
- **Glazing_Area_Distribution:** 144 en cada tipo de distribución, excepto "Sin acristalamiento" con 48 → leve desbalance aceptable.

> ✅ Al no haber categorías dominantes, el modelo no tendrá sesgo por frecuencia. No se necesitan técnicas especiales de balanceo.

🟦🟩 Gráfico de balance de clases (Eficiente)

Qué muestra: cuántos edificios caen en cada clase (Eficiente vs No Eficiente).

Resultado: 384 vs 384 — balance perfecto. Esto es ideal porque el modelo no aprenderá a favorecer una clase por el simple hecho de que hay más de ella en los datos.

3.2 — Análisis de Outliers (valores atípicos)

📦 Boxplots por variable

Qué muestra: un "diagrama de caja" que resume los valores de cada variable. Los puntos fuera de los bigotes son outliers (valores extremos).

Cómo leerlo: la caja central contiene el 50% de los datos. Las líneas (bigotes) llegan hasta 1.5 veces el rango intercuartílico (IQR). Cualquier punto más allá es un outlier.

✅ **Resultado: cero outliers en todas las variables.** El dataset fue generado por simulación computacional (no encuestas reales), por lo que todos los valores son controlados y válidos. No hay que eliminar ni corregir ningún dato.

3.3 — Análisis Bivariado (dos variables a la vez)

📏 Boxplot comparativo de escalas

Qué muestra: todas las variables numéricas en un solo gráfico para comparar sus rangos de valores.

Por qué importa: `Surface_Area` tiene valores entre 514–808, mientras que `Glazing_Area` va de 0 a 0.4. Si no se normalizan, el modelo le dará más "peso" a Surface_Area solo por tener números más grandes, no porque sea más importante.

⚠️ **Conclusión:** hay grandes diferencias de escala. Se debe aplicar **StandardScaler** para poner todas las variables en la misma unidad (media=0, desviación=1).

🎯 Gráficos de dispersión vs Heating_Load

Qué muestra: la relación entre cada variable y el target. Cada punto es un edificio. La línea roja es una regresión lineal simple para ver la tendencia.

Cómo leerlo: si los puntos siguen la línea de cerca → relación fuerte. Si es una nube sin forma → relación débil.

Variable	Relación con Heating_Load	Correlación de Pearson
Overall_Height	Fuerte	0.89 — Los edificios más altos gastan más en calefacción
Roof_Area	Fuerte	−0.86 — Más área de techo = menos gasto (efecto de diseño)
Relative_Compactness	Moderada	Aporta información útil

Surface_Area	**Moderada**	Aporta información útil
Wall_Area	**Moderada**	Aporta información útil
Glazing_Area	**Débil**	Nube de puntos sin patrón claro (pero se mantiene)

🌡️ Mapa de calor de correlaciones (Multicolinealidad)

Qué muestra: la correlación entre *todas* las variables al mismo tiempo. Colores rojos = correlación positiva fuerte, azules = negativa fuerte, blanco = sin relación.

El problema de la multicolinealidad: si dos variables están muy correlacionadas entre sí (ej. > 0.8), le están dando la misma información al modelo. Esto confunde al modelo y puede hacer que los coeficientes no sean confiables.

🚨 **Se detectaron correlaciones problemáticas (|r| > 0.8):**

- Relative_Compactness ↔ Surface_Area: **−0.99** (casi perfecta)
- Relative_Compactness ↔ Roof_Area: −0.87
- Relative_Compactness ↔ Overall_Height: 0.83
- Surface_Area ↔ Roof_Area: 0.88
- Surface_Area ↔ Overall_Height: −0.86
- Roof_Area ↔ Overall_Height: **−0.97**

✂️ **Decisión tomada:**

- Se elimina **Relative_Compactness** (alta correlación con Surface_Area; esta última aporta más al target).
- Se elimina **Roof_Area** (alta correlación con Overall_Height; esta última es más fuerte con el target).

Resumen de decisiones del EDA

🟢 **Sin tratamiento de outliers**

Cero valores atípicos detectados en todas las variables.

- **Sin transformación logarítmica**
  La variable objetivo (Heating_Load) tiene distribución simétrica.
- **Aplicar StandardScaler**
  Las escalas son muy diferentes entre variables numéricas.
- **Eliminar Relative_Compactness**
  Correlación de −0.99 con Surface_Area → información redundante.
- **Eliminar Roof_Area**
  Correlación de −0.97 con Overall_Height → información redundante.
- **Aplicar OneHotEncoder**
  Las variables categóricas (Orientation, Glazing_Area_Distribution) deben convertirse a números para los modelos.

✂️ Sección 4 — División de los Datos

Paso 4 de 7

Se separan los 768 edificios en dos grupos: uno para que el modelo **aprenda** y otro para **evaluar** si realmente aprendió bien.

🏋️ **Entrenamiento (80%)**
614 edificios

El modelo ve estos datos y aprende los patrones.

🧪 **Prueba / Test (20%)**
154 edificios

El modelo *nunca* los vio. Se usan solo para medir qué tan bien predice.

💡 **¿Por qué no usar todos los datos para entrenar?** Si el modelo aprende con todos los datos y se evalúa con los mismos, siempre "sabrá las respuestas". Separar el conjunto de prueba garantiza que la evaluación sea honesta, con datos que el modelo nunca vio.

Balance en el split de clasificación

Conjunto	Eficiente (1)	No Eficiente (0)
Entrenamiento	313 (51%)	301 (49%)
Test	71 (46%)	83 (54%)

El balance se mantiene bien en ambos conjuntos. No hay riesgo de que el modelo aprenda sesgado hacia una clase.

🏭 Sección 5 — Pipeline de Preprocesamiento

Paso 5 de 7

Un **pipeline** es una cadena de transformaciones que se aplican automáticamente a los datos antes de que lleguen al modelo. La ventaja: se aplica exactamente igual a los datos de entrenamiento y de prueba, evitando errores manuales.

Variables que entran al modelo

Numéricas (→ StandardScaler)

- Surface_Area
- Wall_Area
- Overall_Height
- Glazing_Area

Categóricas (→ OneHotEncoder)

- Orientation
- Glazing_Area_Distribution

¿Qué hacen las transformaciones?

📐 StandardScaler — Estandarización de variables numéricas

Problema que resuelve: Surface_Area tiene valores en cientos (514–808), mientras Glazing_Area va de 0 a 0.4. Si los dejamos así, el modelo pensará que Surface_Area es más importante solo por los números grandes.

Cómo funciona: convierte cada variable para que tenga **media = 0 y desviación estándar = 1**. Así todas "compiten" en igualdad de condiciones.

Ejemplo: un edificio con Surface_Area = 671 (la media) se convierte en 0.0. Uno con

808 se convierte en +1.5. Uno con 514 en −1.8.

🔢 OneHotEncoder — Codificación de variables categóricas

Problema que resuelve: los modelos matemáticos no entienden palabras como "Norte" o "Sur". Necesitan números.

Cómo funciona: crea una columna nueva por cada categoría. Si el edificio es "Norte", esa columna vale 1 y las demás 0. Se elimina una categoría (`drop='first'`) para evitar redundancia matemática.

Ejemplo para Orientation: Norte → columna base eliminada; Este → [1,0,0]; Sur → [0,1,0]; Oeste → [0,0,1].

📈 Sección 6 — Modelos de Regresión

Paso 6 de 7

Objetivo: predecir el número exacto de kWh/m² que necesitará el edificio para calefacción (`Heating_Load`). Se comparan tres modelos para ver cuál predice mejor.

Los tres modelos

Modelo	¿Qué es?	Diferencia clave
Regresión Lineal	El modelo más simple. Traza la "mejor recta" a través de los datos.	Sin restricciones. Aprende libremente de los datos.
Ridge (L2)	Regresión lineal con una "penalización" que encoge los coeficientes.	Reduce el sobreajuste castigando coeficientes muy grandes. Todos quedan pequeños pero ninguno llega a 0.
Lasso (L1)	Regresión lineal con una penalización más agresiva.	Puede llevar coeficientes exactamente a 0, actuando como selector automático de variables. Más restrictivo.

Métricas de evaluación

Métrica	¿Qué mide?	¿Mejor valor?
RMSE (Raíz del error cuadrático medio)	Cuántos kWh/m² se equivoca el modelo en promedio. Penaliza más los errores grandes.	Más cercano a 0
MAE (Error absoluto medio)	El error promedio en kWh/m², sin penalizar más los errores grandes. Más intuitivo.	Más cercano a 0
R^2 (Coeficiente de determinación)	Qué porcentaje de la variación del consumo explica el modelo. $R^2=1$ es perfecto, $R^2=0$ no sirve.	Lo más cercano a 1

Resultados

Modelo	RMSE ↓	MAE ↓	R^2 ↑
Regresión Lineal ⭐	**2.98**	**2.23**	**0.9147**
Ridge (L2)	2.99	2.22	0.9144
Lasso (L1)	3.52	2.55	0.8808

🏆 **Mejor modelo de regresión: Regresión Lineal**

Con R^2 = 0.9147, el modelo explica el **91.5% de la variación** en el consumo de calefacción. El error promedio es de solo ~2.98 kWh/m², lo cual es muy bueno dado que el target varía entre 6 y 43 kWh/m².

💡 **¿Por qué Lasso (L1) es el peor aquí?** La regularización agresiva de Lasso está eliminando información útil al poner coeficientes en 0. Con solo 4 variables numéricas relevantes, no sobran variables que descartar, así que el Lasso sacrifica precisión innecesariamente.

Gráficos: Valores Reales vs Predicciones

🎯 Scatter plot real vs predicho (por modelo)

Qué muestra: cada punto es un edificio del conjunto de prueba. El eje X es el valor real de Heating_Load, el eje Y es lo que predijo el modelo. La línea roja punteada diagonal es la "predicción perfecta" (real = predicho).

Cómo leerlo: cuanto más cerca estén los puntos de la línea roja → mejor el modelo. Puntos lejos de la línea = errores grandes.

En Regresión Lineal y Ridge: los puntos siguen muy de cerca la línea diagonal → predicciones muy precisas.

En Lasso: los puntos se dispersan más alrededor de la línea → mayor error promedio.

🏷️ Sección 7 — Modelos de Clasificación

Paso 7 de 7

Objetivo: en lugar de predecir un número exacto, ahora el modelo debe decidir: ¿este edificio es Eficiente o No Eficiente ? Se usa **Regresión Logística** en tres variantes.

> 💡 **¿Por qué "Regresión Logística" si se llama clasificación?** A pesar del nombre, la Regresión Logística es un clasificador. Calcula la *probabilidad* de que un edificio sea eficiente (un número entre 0 y 1), y si esa probabilidad supera 0.5, lo clasifica como Eficiente.

7.1 — Métricas de clasificación

Métrica	Significado en palabras simples	Mejor valor
Accuracy	De todos los edificios, ¿qué porcentaje clasificó correctamente?	→ 1.0 (100%)
Precision	De los edificios que el modelo dijo "son eficientes", ¿cuántos realmente lo son? Evita falsas alarmas.	→ 1.0

Recall	De todos los edificios realmente eficientes, ¿cuántos detectó el modelo? No quiere perderse ninguno.	→ 1.0
F1-Score	Promedio armónico de Precision y Recall. Útil cuando importan ambas métricas por igual.	→ 1.0
ROC-AUC	¿Qué tan bien separa el modelo las dos clases en todos los umbrales posibles? 1.0 = perfecto, 0.5 = igual que adivinar.	→ 1.0

7.2 — Resultados de clasificación

Modelo	Accuracy ↑	Precision ↑	Recall ↑	F1-Score ↑	ROC-AUC ↑
Lasso (L1) ⭐	**0.9805**	0.9595	**1.0000**	**0.9793**	0.9980
Ridge (L2)	0.9740	**0.9589**	0.9859	0.9722	**0.9983**
Sin regularización	0.9610	0.9452	0.9718	0.9583	**0.9983**

🏆 **Mejor modelo de clasificación: Logística Lasso (L1) — por F1-Score**

Accuracy de **98.05%** y Recall perfecto de **100%**: no dejó pasar ningún edificio eficiente sin detectarlo. Solo 3 errores en 154 edificios de prueba.

7.3 — Matrices de confusión

🔲 ¿Qué es una Matriz de Confusión?

Es una tabla 2×2 que muestra en detalle *cómo se equivocó* el modelo. No solo dice cuántos aciertos tuvo, sino también *en qué dirección* se equivocó.

	Pred: No Eficiente	Pred: Eficiente
Real: No Eficiente	TN ✅ Correctamente identificado	FP ❌ Dijo eficiente pero no lo es

	como No Eficiente	(falsa alarma)
Real: Eficiente	FN ❌ Dijo no eficiente pero sí lo era (se lo perdió)	TP ✅ Correctamente identificado como Eficiente

Para el modelo Lasso (L1): Recall = 1.0 significa que los *Falsos Negativos (FN) = 0*. El modelo nunca clasificó un edificio eficiente como no eficiente. Los pocos errores que cometió fueron *Falsos Positivos* (dijo que era eficiente cuando no lo era), que es un error mucho menos costoso en este contexto.

7.4 — Curvas ROC

📉 ¿Qué muestra la curva ROC?

La curva ROC muestra el desempeño del clasificador a *todos los umbrales posibles de decisión* (no solo 0.5). El eje X es la tasa de Falsos Positivos (FPR) y el eje Y la tasa de Verdaderos Positivos (TPR).

Cómo leerla:

- La línea diagonal punteada (AUC = 0.5) representa un clasificador aleatorio, equivalente a lanzar una moneda.
- Cuanto más arriba y a la izquierda esté la curva → mejor el modelo.
- El **AUC (Área Bajo la Curva)** resume todo en un número: más cerca de 1.0 → mejor.

> ✅ **Los tres modelos tienen AUC ≥ 0.998**, muy cerca de la perfección. Las curvas ROC están casi pegadas al eje superior izquierdo, muy lejos de la línea aleatoria. Esto confirma que la separación entre edificios eficientes y no eficientes que aprendieron los modelos es casi perfecta.

✅ Conclusiones Generales

¿Qué aprendimos sobre los edificios?

- La **altura** y el **área de superficie** son los factores más determinantes del consumo de calefacción.
- La **orientación** del edificio tiene un impacto menor pero presente.
- Los edificios más **compactos** (menos superficie exterior por metro cuadrado) tienden a ser más eficientes.
- La cantidad de **ventanales** (Glazing_Area) tiene una relación débil con el consumo, lo cual es sorprendente a priori.

¿Qué aprendimos sobre los modelos?

📈 **Regresión — Mejor: Lineal**

$R^2 = 0.9147$. Predice el consumo exacto con un error promedio de ~3 kWh/m². Simple, interpretable y efectivo.

🏷️ **Clasificación — Mejor: Lasso L1**

Accuracy 98%, Recall 100%. Nunca se perdió un edificio eficiente. Solo 3 errores en 154 edificios.

Aplicación práctica

🏗️ Estos modelos podrían integrarse en una herramienta para arquitectos: dado un diseño propuesto (altura, superficie, orientación, ventanales), el sistema predice automáticamente cuánta energía necesitará y si cumple con estándares de eficiencia **antes de construir**, ahorrando tiempo y dinero en el proceso de diseño.